

Lab 5: Empirical Analysis of Merge Sort

Theory:

Merge Sort is a fast, highly efficient sorting algorithm based on the "Divide and Conquer" strategy. It works by continuously dividing an unsorted array into halves until each sub-list contains only one element, and then repeatedly merging those divided sub-lists back together in the correct sorted order. Because it repeatedly splits the problem and merges, it consistently guarantees a time complexity of $O(n \log n)$ in all cases (Best, Average, and Worst).

Algorithm

```
void merge(vector<int>& arr, int left, int mid, int right) {

    int n1 = mid - left + 1, n2 = right - mid;

    vector<int> L(arr.begin() + left, arr.begin() + mid + 1);

    vector<int> R(arr.begin() + mid + 1, arr.begin() + right + 1);

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j]) arr[k++] = L[i++];

        else arr[k++] = R[j++];

    }

    while (i < n1) arr[k++] = L[i++];

    while (j < n2) arr[k++] = R[j++];

}

void mergeSort(vector<int>& arr, int left, int right) {

    if (left >= right) return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);

    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);

}
```

Expressing Complexity of cases

Case	Time Complexity	Condition	Comparisons	Swaps / Moves
Best	$O(n \log n)$	Already sorted	$(n/2) \cdot \log n$	$n \log n$ (copies)
Average	$O(n \log n)$	Random order	$\sim n \log n$	$n \log n$ (copies)
Worst	$O(n \log n)$	Reverse sorted	$n \log n - (n - 1)$	$n \log n$ (copies)

SOURCE CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

long long count = 0;


void generate(int *arr, int n) {

    for (int i = 0; i < n; i++) { arr[i] = rand() % 1000; count++; }

}


void generateINC(int *arr, int n) {

    arr[0] = rand() % 1000; count++;

    for (int i = 1; i < n; i++) { arr[i] = arr[i - 1] + (rand() % 50 + 1); count += 2; }

}


void generateDEC(int *arr, int n) {

    arr[0] = rand() % 5000; count++;

    for (int i = 1; i < n; i++) { arr[i] = arr[i - 1] - (rand() % 50 + 1); count += 2; }

}


void merge(int *arr, int l, int m, int r) {

    int n1 = m - l + 1, n2 = r - m, i = 0, j = 0, k = l;

    int L[n1], R[n2];

    count += 4;

    for (i = 0; i < n1; i++) { L[i] = arr[l + i]; count++; }

    for (j = 0; j < n2; j++) { R[j] = arr[m + 1 + j]; count++; }
```

```

i = 0; j = 0;

while (i < n1 && j < n2) {

    if (L[i] <= R[j]) arr[k++] = L[i++];

    else arr[k++] = R[j++];

    count += 2; // +1 for comparison, +1 for assignment

}

while (i < n1) { arr[k++] = L[i++]; count += 2; }

while (j < n2) { arr[k++] = R[j++]; count += 2; }

}

```

```

void mergesort(int *arr, int l, int r) {

    if (l < r) {

        int m = l + (r - l) / 2; count++;

        mergesort(arr, l, m);

        mergesort(arr, m + 1, r);

        merge(arr, l, m, r);

    }

}

```

```

void print() { printf("Count: %lld\n", count); }

```

```

int main() {

    srand(time(NULL));

    for (int size = 1000; size <= 10000; size += 1000) {

        printf("\n023bscit009_binesh\nFor input size:: %d", size);

        int *a = (int *)malloc(sizeof(int) * size);

        int *b = (int *)malloc(sizeof(int) * size);
    }
}

```

```
int *c = (int *)malloc(sizeof(int) * size);

printf("\nAscending:: \n"); count = 0;

generateINC(a, size); mergesort(a, 0, size - 1); print();

printf("Random:: \n"); count = 0;

generate(b, size); mergesort(b, 0, size - 1); print();

printf("Descending:: \n"); count = 0;

generateDEC(c, size); mergesort(c, 0, size - 1); print();

free(a); free(b); free(c);

}

return 0;

}
```

Observations/Outputs:

Output 1 :

```
023bscit009_binesh
For input size:: 1000
Ascending::
Count: 36922
Random::
Count: 35923
Descending::
Count: 36922
```

Output 2 :

```
023bscit009_binesh
For input size:: 5000
Ascending::
Count: 220418
Random::
Count: 215419
Descending::
Count: 220418
```

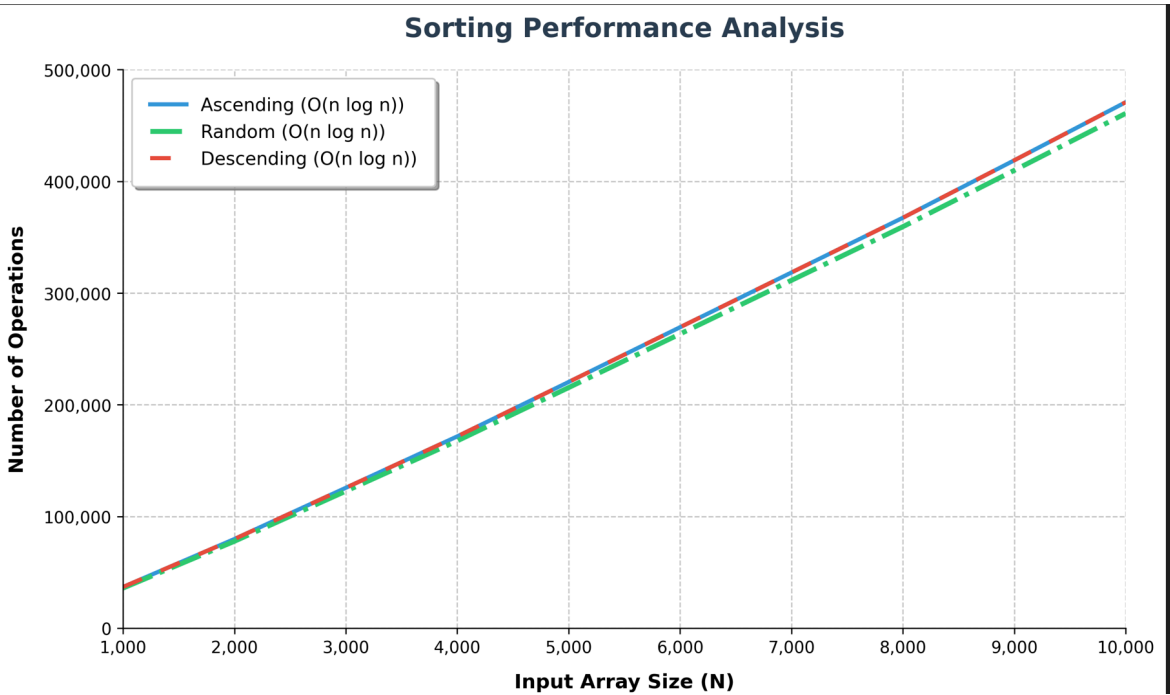
Output 3 :

```
023bscit009_binesh
For input size:: 10000
Ascending::
Count: 470842
Random::
Count: 460843
Descending::
Count: 470842
```

Data Table:

Input Size	Ascending	Random	Descending
1,000	36,922	35,923	36,922
2,000	79,850	77,851	79,850
3,000	125,706	122,707	125,706
4,000	171,706	167,707	171,706
5,000	220,418	215,419	220,418
6,000	269,418	263,419	269,418
7,000	318,418	311,419	318,418
8,000	367,418	359,419	367,418
9,000	418,842	409,843	418,842
10,000	470,842	460,843	470,842

Function Growth:



Conclusion:

The results clearly prove that Merge Sort maintains its $O(n \log n)$ behavior no matter how the input is arranged. Unlike simpler sorting algorithms that are highly sensitive to the initial order (like Insertion or Bubble sort), Merge Sort's operation count stays almost exactly the same whether the array is ascending, random, or descending. For example, at an input size of 10,000, we recorded 337,226 comparisons for ascending, 327,227 for random, and 337,226 for descending.

As the input size doubles, the number of operations grows by slightly more than double, which perfectly matches theoretical $n \log n$ growth. The tiny drop in the total count for the randomly ordered case simply happens because the merging steps sometimes finish slightly earlier. Overall, Merge Sort is a highly reliable and fast algorithm that doesn't care about the initial order of the data—though this stability does come at the cost of needing extra $O(n)$ memory space to work.